

PRÁCTICA 3

Elena Abreu Fernández y Tsan-Yu Wu



**UNIVERSIDAD
DE GRANADA**

1. Limpieza de datos

Como queremos leer los datos en listings.csv sin escribir demasiadas condiciones en el código, usamos la librería OpenCSV que ofrece funcionalidades para leer .csv ficheros.

Además, una vez podemos acceder fácilmente a los datos del CSV nos aseguramos de que los campos que creemos en el índice sólo contengan datos limpios y legibles, eliminando valores basura. Para ello tomamos las siguientes medidas: eliminamos todas las posibles etiquetas HTML con `replaceAll("<[^>]+>", "")` que reemplaza todo el texto que sea de forma `<...>`. También nos aseguramos de cambiar el formato de amenities, que originalmente se guarda como `["Coffee", "Wifi", "Conditioner"]` para que se muestre directamente una lista de forma `Coffee, Wifi, Conditioner`. Por último, revisando los datos se encontraron algunos caracteres extraños como `u2019` que descubrimos que es el valor ASCII del símbolo `'`, concretamente aparecía como un término de amenities : `Pack 'n Play`.

Lo hemos probado con un índice de hasta 1000 documentos y funciona correctamente, aunque para un número mayor a ese nos encontramos problemas de espacio.

2. Creación de campos

En la tipología de **Propiedades** se nos pide trabajar con las siguientes columnas : `id`, `listing_url`, `name`, `description`, `neighborhood_overview`, `neighbourhood_cleansed`, `latitude`, `longitude`, `property_type`, `bathrooms`, `bathrooms_text`, `bedrooms`, `amenities`, `price`, `number_of_reviews`, `review_scores_rating`. Tras analizar la naturaleza de la información hemos decidido indexar estos campos de la siguiente manera:

- **TextField** : `description`, `name`, `neighbourhood_overview`, `bathrooms_text` y `amenities`. Estos campos contienen texto extenso que deseamos que sea analizado y tokenizado, de manera que podamos realizar una recuperación de información eficaz cuando el usuario busque características del complejo, equipamiento, datos del barrio, etc.
- **StringField**: `id`, `neighbourhood_cleansed`, `property_type`. Los añadimos como string al representar identificadores y categorías: `property_type` se ve claramente y `neighbourhood_cleansed` representa categorías pues es el valor automático que le da Airbnb al barrio a partir de la ubicación, es decir, toma valores fijos.
- **IntPoint** : `bedrooms`, `bathrooms` y `number_of_reviews`. Se tratan de campos que toman valores enteros, sobre los que se pueden hacer consultas de igualdad o rango. Para que además luego se muestren dichos valores, los guardamos como **StoredField**.
- **DoublePoint** : `price` y `review_scores_rating`. Se tratan de campos que toman valores decimales, para que además luego se muestren dichos valores, los guardamos como **StoredField**.
- **StoredField** : `listing_url`. No lo necesitamos para las búsquedas pero si queremos que se muestre cuando un usuario acceda a un documento.
- **LatLonPoint** : `latitude` y `longitude`. Permite búsquedas espaciales, los guardamos además como **StoredField** para después mostrarlos.

En la tipología de **Anfitrión** se nos pide trabajar con las siguientes columnas: host_url, host_name, host_since, host_location, host_about, host_response_time, host_is_superhost, host_neighbourhood. Tras analizar la naturaleza de la información hemos decidido indexar estos campos de la siguiente manera:

- TextField: host_location, host_about, host_response_time, host_is_superhost, host_neighbourhood, para que el usuario pueda buscar partes de los contextos que son relevantes en estos campos.
- StringField: host_url, host_name, se tratan como identificadores, porque host_url y host_name deben ser únicos.
- LongPoint: host_since, para almacenar las fechas (como en los ejemplos)
- StoredField: host_since, para que se devuelva en consultas.

3. Análisis

Para decidir qué analizador es más conveniente en nuestro buscador, hemos comparado los resultados obtenidos con StandardAnalyzer, EnglishAnalyzer y WhiteSpaceAnalyzer, intentando simular la experiencia de un usuario real. Al contrario que en prácticas anteriores, hemos descartado directamente el SpanishAnalyzer, pues los datos que indexamos son todos en inglés y carece de sentido aplicar normas del lenguaje español.

Comenzamos aplicando el StandardAnalyzer y observamos los siguientes números en cuanto a términos creados :

Name	Term count	%
host ...	5514	27.62 %
descr...	5121	25.65 %
neigh...	4552	22.80 %
name	1144	5.73 %
id	999	5.00 %
host url	806	4.04 %
amen...	759	3.80 %
host ...	614	3.08 %
host ...	131	0.66 %
host ...	129	0.65 %
neigh...	124	0.62 %
prope...	40	0.20 %
bathr...	20	0.10 %
host ...	12	0.06 %
host ...	2	0.01 %
latitude	0	0.00 %
numbe...	0	0.00 %
price	0	0.00 %
longi...	0	0.00 %
listi...	0	0.00 %
bathr...	0	0.00 %
bedrooms	0	0.00 %
revie...	0	0.00 %
location	0	0.00 %
host ...	0	0.00 %

Las búsquedas son rápidas y efectivas. Recibimos muchos resultados en búsquedas como beach (neighbourhood_overview), center (description) o parking (amenities). Concretamente realiza 200, 67 y 912 hits, respectivamente. El comportamiento es coherente y las recomendaciones son buenas.

Seguimos con el EnglishAnalyzer que genera un número ligeramente menor de términos, pero en ningún caso una diferencia drástica.

Name	Term count	%
host ...	4232	25.42 %
descr...	4065	24.42 %
neigh...	3728	22.39 %
name	1050	6.31 %
id	999	6.00 %
host url	806	4.84 %
ameni...	706	4.24 %
host ...	614	3.69 %
host ...	129	0.77 %
host ...	128	0.77 %
neigh...	124	0.74 %
prope...	40	0.24 %
bathr...	19	0.11 %
host ...	7	0.04 %
host ...	1	0.01 %
latitude	0	0.00 %
numbe...	0	0.00 %
price	0	0.00 %
longi...	0	0.00 %
listi...	0	0.00 %
bathr...	0	0.00 %
bedrooms	0	0.00 %
revie...	0	0.00 %
location	0	0.00 %
host ...	0	0.00 %

La mayor diferencia la observamos en la búsqueda de parking, en la que obtenemos 0 resultados debido al stemming que aplica este analizador. Esto nos lleva a concluir que quizás el stemming es muy agresivo como para asegurar una experiencia de usuario positiva.

Por último, al usar el WhitespaceAnalyzer lo primero que observamos es un aumento significativo del número de términos, cosa que hemos de tener en cuenta pues supone un mayor uso de memoria y menos rapidez:

Name	Term count	%
descr...	9692	30.21 %
host ...	9022	28.12 %
neigh...	7660	23.88 %
name	1865	5.81 %
id	999	3.11 %
ameni...	962	3.00 %
host url	806	2.51 %
host ...	614	1.91 %
host ...	135	0.42 %
host ...	129	0.40 %
neigh...	124	0.39 %
prope...	40	0.12 %
bathr...	20	0.06 %
host ...	12	0.04 %
host ...	2	0.01 %
latitude	0	0.00 %
numbe...	0	0.00 %
price	0	0.00 %
longi...	0	0.00 %
listi...	0	0.00 %
bathr...	0	0.00 %
bedrooms	0	0.00 %
revie...	0	0.00 %
location	0	0.00 %
host ...	0	0.00 %

Además, obtenemos menos recomendaciones con los mismos términos, lo que se puede deber, entre otras cosas, a que este analizador discrimina mayúsculas y minúsculas. Esto hace más frustrante la experiencia de usuario y lleva a resultados inconsistentes.

En conclusión, StandardAnalyzer es la opción más sensata, pues es precisa y normaliza el texto correctamente sin aplicar un stemming agresivo.

4.Trabajo en grupo

Limpieza de datos realizada entre ambas. Tsan-Yu ha realizado la indexación de los campos referidos a Anfitrión y Elena los referidos a Propiedades. La corrección del índice y comparación de analizadores en Luke ha sido realizada por Elena, pues Tsan no puede descargar el programa en su ordenador.

